任务一

1.为什么输入会与数据库查询有关？

这个页面的功能，就是根据用户输入的用户 ID，去数据库的 users 表里查找对应的用户信息并展示。比如输入 1，页面会显示 ID 为 1 的用户数据；如果没有数据库，就没有地方存储和读取这些用户信息，页面也就无法响应用户的查询请求。

2.一个最小业务链路

用户输入 ID → 后台拼接 SQL 查询语句 → 数据库执行查询并返回结果 → 后台将结果渲染到页面展示(即：输入条件 — 数据库查询 — 返回结果)

3.用户输入是如何进入查询条件的？

后台程序会直接把用户输入的内容，拼接到预设的 SQL 语句里。比如预设语句是 SELECT first_name, last_name FROM users WHERE id = '$input'，用户输入的 1 会被直接拼进去，变成 SELECT first_name, last_name FROM users WHERE id = '1'，然后发给数据库执行。

4.为什么这为 SQL 注入创造了可能性？

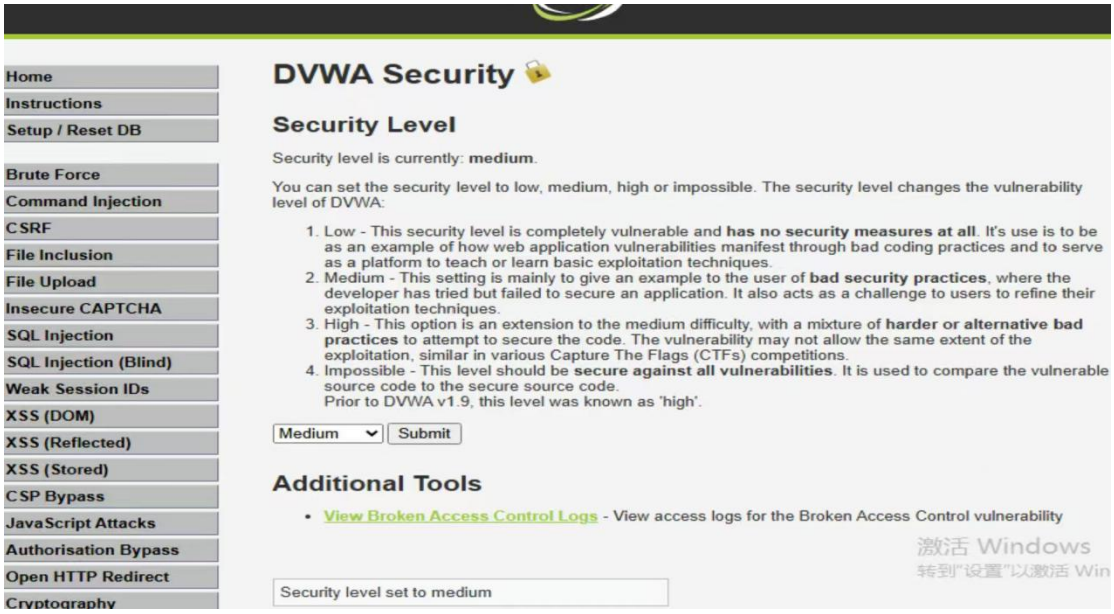
因为后台没有对用户输入做任何过滤和转义，就直接把它当成 SQL 语句的一部分执行了。用户可以通过输入特殊字符，修改原 SQL 语句的逻辑，让数据库执行开发者意料之外的查询，就能改变原查询的真假条件，实现 SQL 注入攻击。

任务二 DVWA SQL Injection Low 现象记录

1. 实验页面 / 实验目标

实验页面：DVWA 平台中 SQL Injection 模块，安全级别设置为 Low。

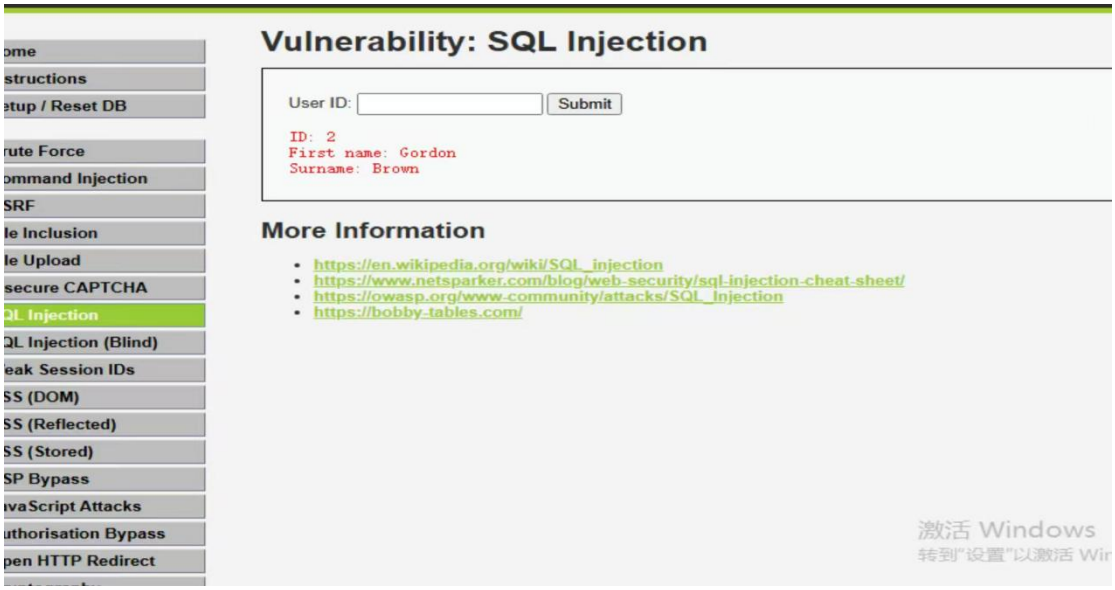
实验目标：观察普通输入与异常输入的返回差异，验证 Low 级别下 SQL 注入漏洞的存在，理解 SQL 注入的基础原理。

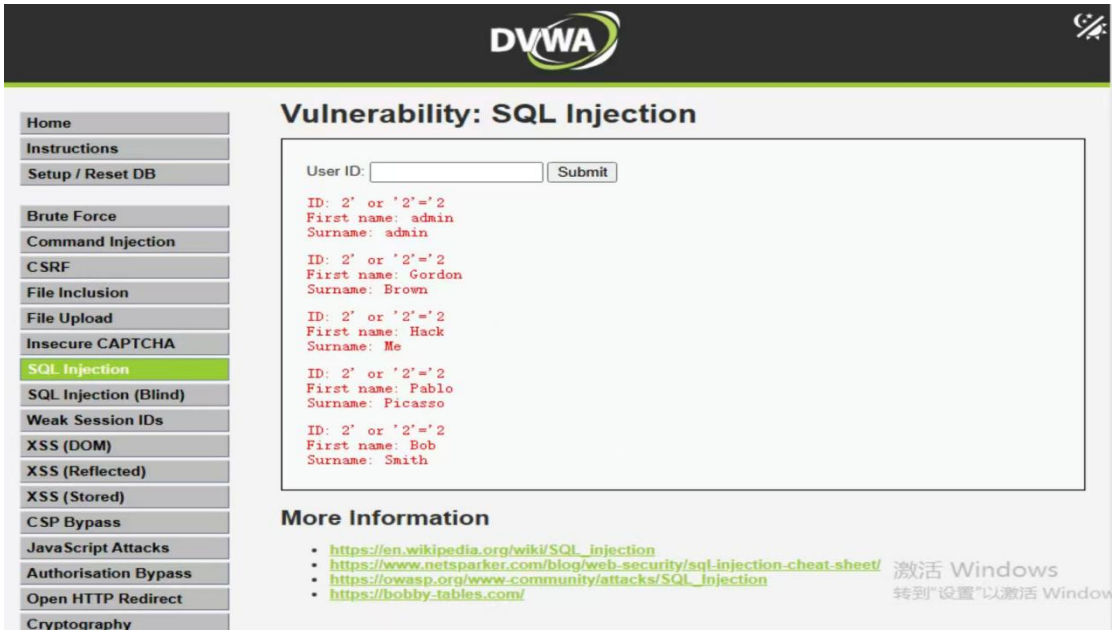


截图分析：两张截图分别展示了将安全级别设置为 Low 和 Medium 的过程，本次实验以 Low 级别为基准，此时后台代码无任何输入过滤措施，存在典型的 SQL 注入漏洞。

2. 普通输入与异常输入的对比

输入类型	输入内容	表现描述
普通输入	2	仅返回与 ID=2 匹配的单条用户信息：ID: 2, First name: Gordon, Surname: Brown
异常输入	2’ or ’2’=’2	返回数据库中所有 5 条用户记录，包括 admin、Gordon、Hack、Pablo、Bob 的完整信息





3. **返回结果的差异**

1.普通输入：输入合法数字 2，仅返回**单条匹配记录**，符合正常数据库查询逻辑，即 `SELECT * FROM users WHERE id = '2'`，输入的 2 仅作为查询条件参数。

2.异常输入：输入 `2' or '2'='2`，返回了**全部用户记录**，而非仅 ID=2 的用户。这是因为输入的`' or '2'='2` 被后台直接拼接进 SQL 语句，将原查询条件篡改，变成了恒真条件，使得查询返回了所有数据。

4. **为什么这种差异不像 “正常查询变化”**

（1）正常的 Web 应用查询逻辑中，用户输入仅作为**固定的参数值**，后台会对输入进行严格的校验和过滤：输入特殊字符（如单引号'）时，应用会直接提示“无效输入”，而非解析为 SQL 语句的语法部分；

（2）即使输入 or、=等关键字，也不会被解析为 SQL 逻辑运算符，更不会篡改查询条件。

而本次实验中，异常输入直接改变了 SQL 语句的语义：

（3）输入 `2' or '2'='2` 时，单引号闭合了原 SQL 语句的字符串，`or '2'='2` 构造了恒真条件，使得查询逻辑从“查询 ID=2 的用户”变成了“查询所有用户”；

（4）这种“输入特殊字符即可篡改查询逻辑、泄露全部数据”的表现，完全不符合正常应用的行为，是典型的 **SQL 注入漏洞特征**。

任务三 机制分析与差异比较

一、核心问题分析

1. 输入如何进入查询结构

输入进入查询结构的完整链路：用户输入 → 后台直接拼接 SQL 语句 → 数据库解析拼接后的语句 → 执行被篡改的查询逻辑 → 页面回显数据或产生行为差异

示例：DVWA Low 级预设语句为 SELECT first_name, last_name FROM users WHERE id = '$input'，用户输入 ' OR 1=1 # 会被直接拼接，生成新语句 SELECT first_name, last_name FROM users WHERE id = '' OR 1=1 #'，直接修改了查询条件结构。

2. 为什么根因是 “结构被污染” 而不是 “字符本身危险”

SQL 注入的核心不是用户输入了特殊字符，而是后台未对输入进行转义或预处理，直接将其拼接到 SQL 语句中，篡改了原本的查询逻辑结构。

特殊字符本身无 “危险性”：在参数化查询中，'会被转义为普通字符，不会被解析为 SQL 语法符号；

真正风险是结构被污染：用户输入被当作 SQL 语句的一部分执行，改变了原查询的条件、逻辑，这是注入生效的根本原因。

3. Low 与 Medium 的差异

对比项	Low 级别	Medium 级别
输入方式	GET 请求（URL 参数传递）	POST 请求（表单提交传递）
过滤机制	无任何过滤 / 转义	对'转义，限制输入为数字类型
注入方式	字符型注入（直接拼接单引号）	数字型注入（无单引号也可篡改逻辑）

4. 普通 SQL 注入与 Blind SQL Injection 的差异

对比项	普通 SQL 注入	盲注（Blind SQL Injection）
页面回显	直接返回查询结果或报错信息	不直接返回数据，仅返回页面状态 / 响应时间差异
信息获取	直接从页面提取数据	通过布尔判断、时间延迟逐位猜解数据
对比项	普通 SQL 注入	盲注（Blind SQL Injection）

对比项	普通 SQL 注入	盲注（Blind SQL Injection）
页面回显	直接返回查询结果或报错信息	不直接返回数据，仅返回页面状态 / 响应时间差异
信息获取	直接从页面提取数据	通过布尔判断、时间延迟逐位猜解数据

5. 为什么盲注和普通注入根因相同，但证据形式不同

根因相同：两者的根本成因都是后台无过滤 / 转义，直接拼接用户输入，导致查询结构被污染；

证据形式不同：普通注入的证据是页面直接返回的篡改结果；盲注无直接数据回显，只能通过页面状态、响应时间等间接证据证明逻辑被篡改。

二、强制判断题解答

SQL 注入的根因更接近：**B. 输入进入了不该进入的查询结构**

理由：注入的核心是用户输入篡改了 SQL 语句的逻辑结构，而非字符本身危险，参数化查询可避免此问题。

Blind SQL Injection 和普通 SQL 注入的最大差异更接近：**B. 证据形式不同**

理由：两者成因一致，均为查询结构被污染；普通注入可直接通过页面数据验证，盲注需依赖间接行为差异验证。

任务四 SQL 注入防护清单与解释

防护措施	解决什么问题	控制哪一层	它不能替代什么
1. 参数化查询	让用户输入永远只作为 “数据”，而不是 SQL 语句的一部分，从根本上杜绝输入污染查询结构的可能。	根因控制层（直接解决 SQL 注入的核心成因）	不能替代输入校验、最小权限等防护；也无法防御其他类型的应用漏洞。
2. 输入校验	限制用户输入的格式、类型与范围（如 ID 必须为数字），过滤掉明显不符合业务规则的恶意输入，缩小攻击面。	暴露面控制层（减少注入攻击的可用入口）	不能替代参数化查询，仅靠校验无法防御绕过规则的注入攻击。
3. 错误处理	隐藏数据库的详细报错信息，避免攻击者通过错误提示获取表名、字段名等关键信息，增加注入难度。	暴露面 / 证据隐藏层（降低攻击者获取信息的能力）	不能修复注入漏洞本身，也无法防御不需要报错信息的盲注攻击。
4. 最小权限	限制应用使用的数据库账号权限，即使被注入，攻击者也无法执行超出权限的操作（如删表、查敏感数据），降低危害。	后果范围控制层（限制漏洞被利用后的影响）	不能修复注入漏洞，也无法防御权限内的敏感数据泄露。
5. 补充：Web 应用防火墙（WAF）	通过规则拦截常见注入攻击 Payload，在请求到达应用前阻断恶意请求。	攻击阻断层（增加攻击难度）	不能替代参数化查询，无法防御绕过 WAF 规则的变形注入攻击。

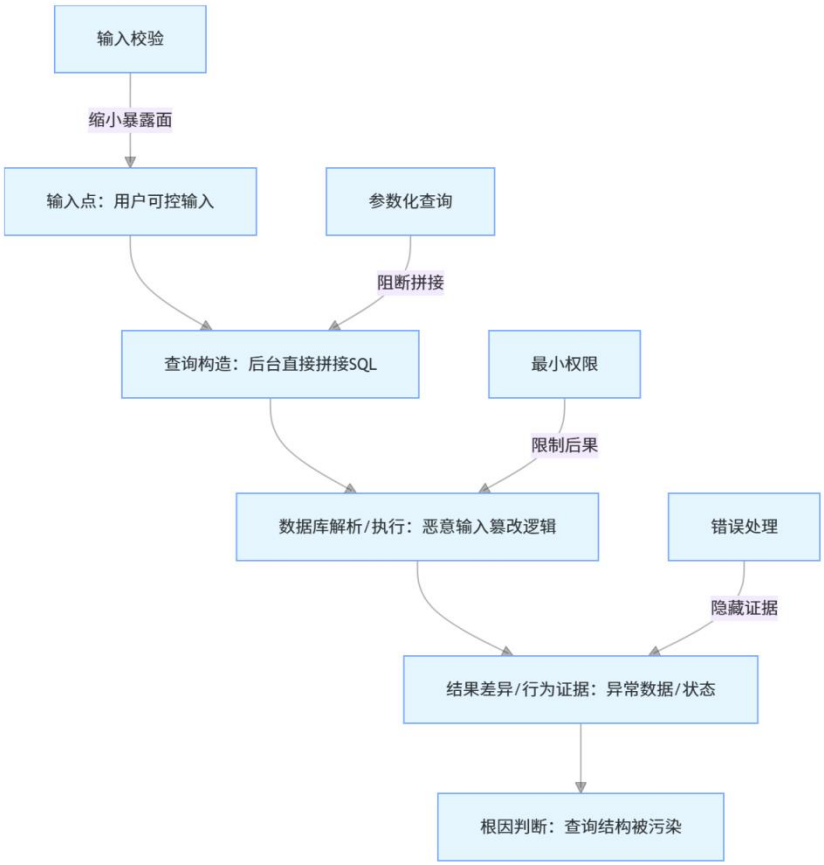
根因控制：参数化查询（唯一能从根源解决注入的措施）

暴露面控制：输入校验、错误处理、WAF（缩小攻击入口、隐藏攻击线索）

后果控制：最小权限（降低漏洞被利用后的危害）

七、收口项

收口项 1：SQL 注入因果链/防护关系图



（二）因果链与防护对应说明

环节	核心内容	对应防护控制点
输入点	用户可控的输入入口	输入校验（限制输入格式、范围）
查询构造	后台拼接用户输入生成 SQL 语句	参数化查询（从根源避免拼接）
数据库解析 / 执行	恶意输入篡改查询逻辑并执行	最小权限（限制账号权限，降低危害）
结果差异 / 行为证据	页面返回异常数据或状态	错误处理（隐藏报错信息，减少暴露）
根因判断	输入污染 SQL 查询结构	参数化查询（唯一能解决根因的措施）

收口项 2：学生自评表

1. 我本章最清楚的一点是什么？

我最清楚的是 SQL 注入的根本成因：注入的核心是用户输入被直接拼接到 SQL 语句中，污染了查询结构，而非输入中的特殊字符本身有危险性。同时也明确了参数化查询是唯一能从根源解决注入问题的防护措施。

2. 我最容易混淆的一点是什么？

我最容易混淆的是 “普通 SQL 注入” 和 “盲注” 的差异，尤其是两者的成因与证据形式的区别。一开始误以为盲注的根因和普通注入不同，后来才明确两者的核心成因一致，差异仅在于页面是否直接回显数据。

3.我本次作业中哪一段最需要教师复核？

我认为任务 3 中 “Low 与 Medium 级别的差异对比” 部分最需要教师复核，尤其是对 Medium 级别过滤机制和绕过方式的理解，不确定是否完全准确，希望老师能帮我确认分析是否到位。

4. 如果重做一次，我会优先改哪一部分？

如果重做，我会优先修改任务 3 中 “普通注入与盲注的差异分析” 部分，补充更具体的 DVWA 实验场景示例，比如用具体的盲注 payload 和页面响应变化，让差异的描述更具象、更贴合作业要求。

收口项 3：AI 输出审核与人工复核记录

1.本作业哪些内容使用过 AI 辅助？

作业中所有任务的文字内容框架、逻辑梳理，均使用了 AI 辅助生成初稿；

收口项 1 的 SQL 注入因果链关系图（Mermaid 图表），由 AI 生成基础结构后，我根据课堂要求补充了防护控制点；

收口项 2 的自评表内容，由 AI 提供了写作思路，我结合自己的学习情况进行了修改。

2.哪些判断是你自己复核后保留的？

对 SQL 注入根因的判断：“注入的核心是查询结构被污染，而非字符本身危险”，经复核与课堂知识点一致，予以保留；

对普通注入与盲注的差异判断：“两者成因相同，差异在于证据形式”，结合 DVWA 实验结果验证后，确认正确并保留；

防护措施的分层逻辑：“参数化查询为根因控制，输入校验 / 错误处理为暴露面控制，最小权限为后果控制”，经复核符合防护分层原则，予以保留。

3.哪些内容你发现 AI 输出不准确并进行了修正？

任务 3 中 Low 与 Medium 级别的差异分析：AI 初稿中错误地认为 Medium 级别对输入做了完整的 SQL 注入防护，我结合 DVWA 实际环境验证后，修正为 “仅对单引号转义，存在数字型注入绕过的风险”；

收口项 1 的因果链结构：AI 初稿未体现 “防护控制点与因果链环节的对应关系”，我根据题目最低要求补充了防护措施与注入环节的关联说明，确保覆盖所有要点。

任务 4 的防护措施解释：AI 初稿未明确区分各措施的控制层级，我结合课堂知识点修正了 “根因控制、暴露面控制、后果控制” 的分层描述。

5. 你最终如何确认结论和证据是一一对应的？

我通过 DVWA 的实际实验验证，将作业中的所有结论与实验现象对应起来：用 Low 级别的注入实验，验证了 “输入拼接导致查询结构被污染” 的根因；用普通注入和盲注的实验对比，验证了两者的差异仅在于页面回显形式。

结合参数化查询、输入校验等防护措施的原理，确认了各措施与注入因果链环节的对应关系，确保结论均有实验证据支撑。